Experiment 1: Case Study on the Analysis of SDLC Models Applied to a Real-World Software Application

Aim

To analyze the Software Development Life Cycle (SDLC) models used in the development of a real-world software application and identify the most suitable SDLC model.

Objective

- Study different SDLC models.
- Analyze the development process of a real-world software application.
- Compare the advantages and limitations of the various SDLC models.
- Recommend the most appropriate SDLC model for the application.

Selected Software Application

Online Food Delivery System (e.g., Swiggy / Zomato)

Overview

An online food delivery system lets customers register or log in, search restaurants, order food, pay online, track the order and leave ratings and reviews.

Functional Requirements

Module	Functions
Customer	Registration, Login, Search Food, Add to Cart, Place Order, Online Payment, Order Tracking
Restaurant	Manage Menu, Accept/Reject Orders, Update Order Status
Delivery	Accept Delivery, Live Location Tracking, Delivery Confirmation
Admin	User Management, Restaurant Management, Reports, Analytics

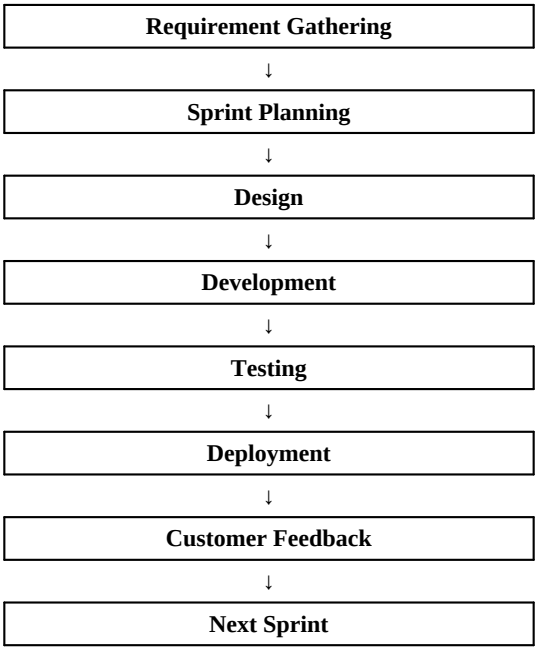
SDLC Models Analysis

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Simple and easy to manage	Difficult to accommodate changing requirements
Prototype	Medium	Better understanding of user requirements	Increased development cost
Spiral	High	Good risk management	Expensive and complex
Agile	Very High	Continuous improvement, customer feedback	Needs an experienced team
DevOps	Very High	Continuous integration and deployment	Infrastructure complexity

Why Agile is Best?

- Customer requirements change frequently.
- New restaurant features are added regularly.
- Bug fixes must be released quickly.
- Mobile apps need continuous updates.
- Frequent customer feedback improves the product.

Agile Development Process



SDLC Phases

1. Requirement Analysis

Customer requirements, restaurant requirements, payment gateway and GPS tracking are studied. **Deliverable:** Software Requirement Specification (SRS).

2. System Design

Database design, UI design, architecture design and API design. **Deliverables:** UML diagrams, database schema.

3. Development

Modules developed: Login, Order, Payment and Delivery. Technologies: Java/Python, React, Node.js with MySQL/PostgreSQL.

4. Testing

Unit, Integration, System and User Acceptance Testing (UAT).

Test Case	Expected Result
Login	Successful login
Invalid password	Error message
Place order	Order created
Payment	Payment successful

5. Deployment

Cloud deployment (AWS, Azure, Google Cloud), web server deployment and mobile app release.

6. Maintenance

Bug fixes, security updates, feature enhancements and performance optimization.

Comparison of SDLC Models

Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Requirement changes	Poor	Good	Excellent	Excellent	Excellent
Customer feedback	Low	High	High	Very High	Very High
Risk handling	Low	Medium	High	Medium	High
Delivery speed	Slow	Medium	Medium	Fast	Very Fast
Cost	Low	Medium	High	Medium	Medium
Suitable for food app	No	Partially	Yes	Best	Best

Results

- The food delivery system needs continuous feature updates and customer feedback.
- Agile supports iterative development, quick releases and flexibility.
- DevOps complements Agile through continuous integration, testing and deployment.
- Hence **Agile with DevOps practices** is the most suitable approach.

Conclusion

Selecting an SDLC model depends on project requirements. For dynamic applications such as online food delivery, Agile gives flexibility, faster delivery and continuous customer involvement, while DevOps adds deployment automation and operational efficiency. Together they ensure high software quality, rapid feature delivery and better user satisfaction.